

Design and Implementation of a Compiler for COOL (Classroom Object-Oriented Language)

Thesis submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology
in
Computer Science and Engineering

Submitted by

SURAJ KUMAR
CS21B1028

Under the supervision of

Dr. Ramesh Kumar Jallu



Department of Computer Science and Engineering
Indian Institute of Information Technology Raichur - 584135
Karnataka, India

May 2025

CERTIFICATE OF RECOMMENDATION

This is to certify that the work embodied in this thesis entitled **Design and Implementation of a Compiler for COOL (Classroom Object-Oriented Language)** has been satisfactorily completed by **Mr. SURAJ KUMAR** (Roll No. CS21B1028 - 2021 - 2025). This is to certify that the work embodied in this project report is a bonafide piece of work carried out under my supervision and guidance at the Indian Institute of Information Technology Raichur, in partial fulfillment of the requirements for the award of the **Bachelor of Technology** degree in **Computer Science and Engineering** from the **Indian Institute of Information Technology Raichur**, during the academic year 2021 - 2025.

Dr. Ramesh Kumar Jallu,
Department of Computer Science and Engineering,
Indian Institute of Information Technology Raichur, Karnataka, India.
(Supervisor)

Forwarded By:

Dr. Ramesh Kumar Jallu,
Asso. HoD, Department of Computer Science and Engineering,
Indian Institute of Information Technology, Raichur, Karnataka.

CERTIFICATE OF APPROVAL

This is to certify that the thesis entitled **Design and Implementation of a Compiler for COOL (Classroom Object-Oriented Language)** is a bonafide record of work carried out by **SURAJ KUMAR** (CS21B1028) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** degree of the Department of Computer Science and Engineering, during the period of **2021** to **2025**. It is understood that by this approval, the undersigned do not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the thesis only for the purpose of which it has been submitted.

Signature of Examiner 1

Date:

Signature of Examiner 2

Date:

DECLARATION

I hereby declare that the thesis entitled **Design and Implementation of a Compiler for COOL (Classroom Object-Oriented Language)** contains a literature survey and original research work by the undersigned candidate, as a part of his degree of **Bachelor of Technology in Computer Science and Engineering, Indian Institute of Information Technology Raichur**. All the information have been obtained and presented in accordance with academic rules and ethical conduct.

I also declare that, as required by these rules and conduct, I have fully cited and referenced all materials and results that are not original to this work.

Name: Suraj Kumar

Roll No.: CS21B1028

Thesis Title: Design and Implementation of a Compiler for COOL (Classroom Object-Oriented Language)

Signature of the Candidate:

Abstract

This thesis presents the Design and implementation of a compiler for COOL (Classroom Object-Oriented Language), a language introduced to help students understand core principles of compiler construction and object-oriented programming. The compiler processes COOL programs through all major stages of compilation: lexical analysis, syntax analysis (parsing), semantic analysis, intermediate representation generation, and back-end code generation targeting MIPS assembly. Each stage of the compiler pipeline is developed with careful attention to correctness, extensibility, and alignment with standard compiler design practices.

The motivation for this work originates from a strong passion for low-level programming and system software. With a long-term ambition to contribute to companies such as Intel, AMD, Qualcomm, or NVIDIA—leaders in processor and hardware development—this project serves as a foundational step. These companies rely heavily on efficient compilers and language infrastructure to build device drivers, system tools, and optimized software for their hardware platforms. Understanding how compilers work from the ground up is thus crucial for meaningful contributions in such environments.

One of the key highlights of this compiler is its handling of COOL-specific language constructs, such as `let` expressions for scoped variable declarations, `case` expressions for type-based branching, and a strong emphasis on class inheritance and polymorphism. These features were implemented with a deep focus on semantic correctness, memory layout management, and runtime behavior consistency.

The project was developed with a strong emphasis on modularity and maintainability. Lexer rules were defined using Flex, grammar rules using Bison, and the code generator was manually written to emit MIPS assembly compatible with the SPIM simulator. In addition to theoretical understanding, the project required debugging complex semantic errors, managing memory layouts for objects, generating dispatch tables, and simulating runtime stack and heap environments—all of which contributed to building a deep understanding of systems programming.

Ultimately, this thesis showcases not just a working compiler, but also a hands-on journey through all layers of language translation, guided by curiosity, ambition, and a desire to work closely with the technologies that power modern computing.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Objectives	5
1.3	Overview of COOL Language	5
1.3.1	COOL Language and Stanford Infrastructure	6
1.3.2	Key Features	6
1.3.3	Language Structure	6
1.3.4	Type System	7
1.3.5	Expressions	7
1.3.6	Basic Classes	8
1.3.7	Operational Semantics	9
1.3.8	Compiler and Runtime	9
1.3.9	Conclusion	10
2	Literature Review	11
2.1	Overview of Compiler Design	11
2.1.1	Compiler Phases	11
2.1.2	Frontend and Backend	11
2.1.3	Error Handling and Runtime Support	12
2.2	Related Work	12
2.2.1	Compiler Design Literature	12
2.2.2	Runtime Support for COOL	12
2.2.3	Comparison with Related Educational Compilers	12
2.3	Distinctive Features of COOL	12
2.3.1	Distinctive Features of COOL	12
3	System Design	15
3.1	Architecture of the COOL Compiler	15
3.1.1	High-Level Components	15
3.1.2	Modular Design	15
3.1.3	Class Hierarchy and AST Design	16
3.1.4	Backend Selection and Output	16
3.2	Phase-wise Design (Lexing to Code Generation)	16
3.2.1	Lexical Analysis (Lexer)	16
3.2.2	Syntax Analysis (Parser)	17
3.2.3	Semantic Analysis	17
3.2.4	Intermediate Representation (IR) Generation	17
3.2.5	Code Generation	17
3.2.6	Runtime Integration	18

3.2.7	Phase Coordination	18
3.3	Tools and Technologies Used	18
3.3.1	Tools and Technologies Used	18
4	Implementation	20
4.1	Lexical Analyzer (Scanner)	20
4.1.1	Introduction	20
4.1.2	Structure of the Flex File	20
4.1.3	Key Components of <code>cool.flex</code>	20
4.1.4	Token Generation and Semantic Values	21
4.1.5	Error Handling	21
4.1.6	Integration with the COOL Compiler	22
4.1.7	Improvements and Extensions	22
4.1.8	Conclusion	22
4.2	Parser Design in COOL	22
4.2.1	Overview	22
4.2.2	Parser Components	22
4.2.3	Semantic Value Union	22
4.2.4	Token Definitions	23
4.2.5	Grammar and Productions	23
4.2.6	Handling Classes and Features	23
4.2.7	Expression Parsing	23
4.2.8	LET and CASE Expressions	24
4.2.9	Block and Dispatch Handling	24
4.2.10	Operator Precedence	24
4.2.11	Location Tracking and Line Numbers	24
4.2.12	Error Handling	25
4.2.13	Summary	25
4.3	Semantic Analyzer	25
4.4	Introduction	25
4.5	Core Data Structures	25
4.5.1	Symbol Management	25
4.5.2	Type Environment	26
4.6	Class Hierarchy Validation	26
4.6.1	Basic Class Installation	26
4.6.2	Inheritance Validation	26
4.7	Method Environment Construction	26
4.7.1	Global Method Environment[M]	26
4.7.2	Method Resolution	26
4.8	Type Checking Framework	27
4.8.1	Subtyping Relations	27
4.8.2	Type Join Operations	27
4.9	Expression Type Checking	27
4.9.1	Introduction	27
4.9.2	Assignment Expressions	28
4.9.3	Method Dispatch	28
4.9.4	Conditional Expressions	31
4.9.5	Case Expressions	31

4.10	Scope Management	33
4.10.1	Object Environment Initialization	33
4.10.2	Scope Handling	33
4.11	Method Validation	33
4.12	Attribute Management	33
4.13	Special Type Handling	33
4.13.1	SELF_TYPE Semantics	33
4.13.2	Primitive Type Operations	33
4.13.3	Arithmetic Operation	34
4.13.4	Boolean Operation	34
4.14	Constant Expressions	34
4.15	Special Expressions	35
4.15.1	Object Creation	35
4.15.2	Void Check	35
4.15.3	Empty Expression	35
4.16	Error Reporting	35
4.17	Analysis Phases	36
4.17.1	Class Table Construction	36
4.17.2	Method Environment Building	36
4.17.3	Type Checking Phase	36
4.18	Integration with Compilation Pipeline	36
4.19	Code Generation (MIPS)	36
4.19.1	Introduction	36
4.19.2	Overall Architecture	36
4.19.3	Prototype Object Installation	39
4.19.4	Method Code Generation: <code>install_methods</code>	42
4.19.5	Object Initializer Method	44
4.19.6	Class Representation	46
4.19.7	Environment Tracking	46
4.19.8	Explanation of Components	47
4.20	Code Generation Process	48
4.20.1	Initialization	48
4.20.2	Class Table Construction	48
4.20.3	e.g III) Object Initialization	49
4.21	Expression Code Generation	50
4.21.1	e.g IV) Object Creation(new syntax)	51
4.22	Challenges and Solutions	51
4.23	Conclusion	52
5	Testing And Challenges In COOL Compiler Implementation	53
5.1	Testing	53
5.2	Challenges	53
6	Conclusion and Future Work	55
6.1	Summary of Achievements	55
6.2	Future Improvements	56
	References	59

A Grammar Rules	60
A Sample Programs	62
A.1 Binary Search Tree	62
A.2 Linked List	64
A.3 Bubble Sort	66

Introduction

1.1 Motivation

My primary motivation for undertaking this project stems from a deep interest in low-level programming and systems software. I have always been fascinated by how computers work at the architectural and hardware-interfacing level, and this curiosity naturally led me to explore fields like compilers, assembly languages, and operating system internals.

I aspire to work at industry-leading companies such as Intel, AMD, Qualcomm (Snapdragon), and NVIDIA—organizations that are at the forefront of processor design and low-level system optimization. These companies develop cutting-edge hardware and require robust software, such as device drivers and firmware, that interact closely with the underlying architecture. I realized that a strong understanding of compiler design is an essential stepping stone toward this goal, as compilers are central to translating high-level logic into hardware-executable instructions.

Initially, I was intrigued by the idea of how high-level code is transformed into machine-level instructions. This curiosity encouraged me to learn how compilers work from scratch. I began by studying the theoretical foundations of compiler design, including lexical analysis, parsing, semantic analysis, and code generation. Over time, this learning journey evolved into a hands-on project where I attempted to build a working compiler for the COOL (Classroom Object-Oriented Language).

By understanding both traditional and modern approaches to compiler design, I aimed to bridge the gap between theoretical concepts and practical system-level development.

Ultimately, this project is not just an academic exercise, but a significant step towards realizing my long-term goal of contributing to system software development for high-performance hardware platforms. By learning to build compilers and understanding the toolchains used in real-world development, I believe I am better equipped to contribute to low-level software domains, including device drivers, embedded systems, and operating systems.

1.2 Objectives

1.3 Overview of COOL Language

The COOL is a small, statically typed object-oriented programming language designed by Alex Atkin, who is currently a professor at Stanford University. COOL retains many features of modern programming languages.

1.3.1 COOL Language and Stanford Infrastructure

The COOL programming language was introduced by Alex Atkin, he is professor at Stanford University. It is designed for teaching object-oriented programming and compiler design. Our project builds on this educational infrastructure by implementing the full COOL compiler pipeline from scratch, including lexical analysis, parsing, semantic analysis, and backend code generation OF MIPS. For a comprehensive understanding of the COOL language, readers are encouraged to consult the official COOL manual provided as part of the Stanford COOL framework [1].

1.3.2 Key Features

Cool incorporates several important programming language concepts:

- Object-oriented programming with classes and inheritance
- Static typing with type safety guarantees
- Expression-based semantics
- Information hiding through method-based access control means Encapsulaton.
- Supports polymorphims eg., case construct of COOL language is prime example that illustrates polymorphism. Also it supports method overriding but not pure virtual function.

1.3.3 Language Structure

Program Organization Cool programs are organized into classes. Every program must contain a `Main` class with a `main` method that takes no parameters. Program execution begins by evaluating `(new Main).main()`.

Classes Classes are the fundamental building blocks of Cool programs. A class definition has the following syntax:

```
class <type> [inherits <type>] {  
    <feature_list>  
};
```

Key characteristics of Cool classes:

- Class names begin with uppercase letters
- Single inheritance only (no multiple inheritance)
- Inheritance forms a tree with `Object` as the root
- Classes cannot be redefined

Features Class bodies contain features, which can be either attributes (variables) or methods:

```
-- Attribute definition
<id> : <type> [<- <expr>];

-- Method definition
<id>(<id> : <type>, ..., <id> : <type>) : <type> { <expr> };
```

Feature names begin with lowercase letters. Attributes have class-local scope, while methods have global scope.

1.3.4 Type System

Cool has a static type system that guarantees type safety at compile time.

Type Hierarchy

- Every class defines a type
- The type hierarchy mirrors the inheritance hierarchy
- Special type `SELF_TYPE` for polymorphic return types
- Basic classes: `Object`, `IO`, `Int`, `String`, `Bool`

Type Conformance Type conformance ($\leq$) is defined as:

- $A \leq A$ for all types A (reflexive)
- If C inherits from P , then $C \leq P$
- If $A \leq C$ and $C \leq P$ then $A \leq P$ (transitive)

Type Checking The type checker:

- Infers a type for every expression
- Guarantees no runtime type errors
- May reject some safe programs to maintain soundness

The static type of an expression may be more general than its dynamic type, but never more specific ($De \leq Se$ for all expressions e).

1.3.5 Expressions

Cool is an expression language - most constructs are expressions that return values.

Common Expressions

- **Constants:** `true`, `false`, integers, strings
- **Identifiers:** Variables, parameters, `self`
- **Assignment:** `id <- expr`
- **Dispatch** (method call): Three forms:
 - `expr.f(expr1, ..., exprn)` (dynamic dispatch)
 - `f(expr1, ..., exprn)` (shorthand for `self.f(...)`)
 - `expr@A.f(expr1, ..., exprn)` (static dispatch)

Control Structures

- **Conditionals:**
`if expr then expr else expr fi`
- **Loops:**
`while expr loop expr pool`
- **Blocks:**
`{ expr1; ...; exprn; }`

Other Expressions

- **Let** (local variables):
`let id1 : type1 [<- expr1], ..., idn : typen [<- exprn] in expr`
- **Case** (runtime type test):
`case expr of
 id1 : type1 => expr1;
 ...
 idn : typen => exprn;
esac`
- **Object creation:** `new type`
- **Type test:** `isvoid expr`

1.3.6 Basic Classes

Cool provides several built-in classes:

Object The root of the class hierarchy with methods:

- `abort()` : `Object` - halts execution
- `type_name()` : `String` - returns class name
- `copy()` : `SELF_TYPE` - creates shallow copy

IO Provides basic input/output:

- `out_string(x : String) : SELF_TYPE`
- `out_int(x : Int) : SELF_TYPE`
- `in_string() : String`
- `in_int() : Int`

Primitive Types

- **Int**: 32-bit integers (default 0)
- **String**: Character strings with methods:
 - `length() : Int`
 - `concat(s : String) : String`
 - `substr(i : Int, l : Int) : String`
- **Bool**: Boolean values `true` and `false` (default `false`)

1.3.7 Operational Semantics

Cool's execution model involves:

- **Environment**: Maps identifiers to locations
- **Store**: Maps locations to values (objects)
- **Self object**: The object on which a method was invoked

Objects contain:

- Class name
- Attributes (with their locations)
- Methods (inherited from class definition)

1.3.8 Compiler and Runtime

The Cool compiler (`coolc`) compiles Cool source files (extension `.cl`) to MIPS assembly code (extension `.s`). Programs are executed using the SPIM MIPS simulator.

Compilation Process

1. Multiple `.cl` files are concatenated
2. Each file must contain complete class definitions
3. Output is MIPS assembly code

Runtime Errors Cool programs can generate runtime errors for:

- Dispatch on `void`
- Case on `void`
- No matching case branch
- Division by zero
- Substring out of range
- Heap overflow

1.3.9 Conclusion

Cool provides a compact but complete object-oriented language, and it is also a Turing Complete Language. Its design incorporates:

- A clean object model with single inheritance
- A sound static type system
- Automatic memory management
- Comprehensive semantics covering both typing and execution

Turing Complete Languages A **Turing complete language** is a programming language (or formal system) that is powerful enough to simulate any Turing machine. This implies that it can perform any computation that can be described algorithmically, provided it has sufficient time and memory resources.

Literature Review

2.1 Overview of Compiler Design

A compiler is a complex software system that translates high-level programming code into low-level machine or intermediate code that can be executed by a computer. The process involves multiple well-defined stages, each responsible for transforming and analyzing the source program while preserving its semantics and optimizing its execution.

The classical structure of a compiler is divided into two major parts: the **frontend** and the **backend**. The frontend is responsible for analyzing the syntax and semantics of the source code, while the backend focuses on optimization and target code generation. The intermediate representation (IR) serves as a bridge between these two components.

2.1.1 Compiler Phases

The compiler for the COOL language follows the standard design pipeline consisting of the following phases:

1. **Lexical Analysis (Scanner):** This phase converts the stream of characters from the source code into a stream of tokens. It removes whitespace and comments and recognizes keywords, identifiers, symbols, and constants.
2. **Syntax Analysis (Parser):** The parser takes the token stream and constructs a parse tree (or abstract syntax tree, AST) based on the grammar rules of the language. It checks for syntactic correctness.
3. **Semantic Analysis:** This phase verifies semantic consistency, such as type checking, scoping rules, method resolution, and inheritance correctness. The symbol table is constructed and maintained to track variable and method definitions across scopes.
4. **Code Generation:** Finally, the AST is converted into target code, that is MIPS assembly, suitable for execution.

2.1.2 Frontend and Backend

The frontend ensures the program is well-formed and valid in the context of the COOL language, while the backend is responsible for translating the program into a low-level representation that can be executed. This thesis supports **MIPS backend**.

2.1.3 Error Handling and Runtime Support

Each phase is equipped with error detection and reporting mechanisms. During code generation, runtime support functions (e.g., `abort`, `String.length`, `IO.out_int`) are linked into the final output to ensure consistent and correct runtime behavior.

This structured pipeline ensures that the compiler is modular, extensible, and maintainable while enabling sophisticated backend features like multiple code targets.

2.2 Related Work

The development of this COOL (Classroom Object-Oriented Language) compiler draws upon several key areas of prior work, including compiler design methodologies, educational compiler infrastructure, code generation techniques, and runtime support systems.

2.2.1 Compiler Design Literature

The structure and design of the compiler were influenced heavily by classical and advanced compiler textbooks. The “Dragon Book” by Aho, Lam, Sethi, and Ullman [2] provided the foundational knowledge required to implement various compiler phases such as lexical analysis, syntax parsing, semantic checking, and intermediate representation generation. Further insight into advanced compiler mechanisms, including scope management, control-flow optimization, and runtime environment organization, was derived from Muchnick’s “Advanced Compiler Design and Implementation” [3].

2.2.2 Runtime Support for COOL

Runtime support functions such as `abort`, `out_string`, `in_int`, `str_concat`, `str_length`, and `str_substr` were based on the runtime interface defined in the Stanford COOL framework [1]. These functions enable key operations like input/output, string manipulation, and error handling. They were implemented in accordance with both the MIPS.

2.2.3 Comparison with Related Educational Compilers

While there exist other educational compilers such as MiniJava and Decaf, COOL provides a richer object-oriented model with inheritance, dynamic dispatch, and runtime type checking.

2.3 Distinctive Features of COOL

2.3.1 Distinctive Features of COOL

The COOL is a pedagogical programming language designed at Stanford University to teach the principles of compiler construction and object-oriented programming. Despite its minimal syntax and simple semantics, COOL incorporates several features that mirror those in real-world object-oriented languages like Java and C++, while being tailored for learning purposes.

Among its distinctive features, two stand out for their expressive power and implementation complexity: the `let` expression and the `case` expression.

The **let** Expression

The **let** expression in COOL is used for local variable declarations and supports optional initialization. It introduces a new scope for each declared identifier and allows variables to be initialized with expressions or left uninitialized (defaulting to type-specific values such as 0 for `Int` or `void` for class types).

```
let x : Int <- 5, y : Bool in x + 1
```

This construct is particularly notable for:

- Supporting multiple variable declarations in a single expression.
- Allowing both initialized and uninitialized variables.
- Creating a nested scope dynamically at runtime, which must be carefully managed during semantic analysis and code generation.

From a compiler design perspective, the **let** expression necessitates:

- The creation of a new scope frame in the symbol table.
- Type checking of optional initialization expressions.
- Cleanup of bindings after the **let** block completes.
- Emission of code that handles dynamic memory (for classes) or stack-based storage (for primitives).

The **case** Expression

The **case** expression in COOL provides a form of dynamic dispatch or pattern matching based on the runtime type of an expression. It allows for type-safe multi-way branching and is conceptually similar to a type-based switch statement.

```
case e of
  x : Int => e1;
  y : Bool => e2;
  z : Object => e3;
esac
```

Key characteristics include:

- Branches are ordered by specificity (most derived class matched first).
- Runtime type of the expression `e` determines the branch to execute.
- Each branch introduces a new variable in its own scope.

Compiler implications:

- Requires the runtime system to determine the dynamic type tag of the expression.
- The compiler generates code that performs a sequence of type comparisons and jumps to the correct branch.

- Branch order and inheritance hierarchy must be considered to avoid ambiguity.
- The inferred type of the entire **case** expression is the least upper bound (LUB) of all branch result types.

This mechanism allows COOL to support flexible and type-safe polymorphic behavior without requiring explicit downcasting.

Other Notable Features

While **let** and **case** are especially unique, COOL also includes:

- **Strong Static Typing:** Type checking is done at compile-time, preventing many runtime errors.
- **Single Inheritance:** A well-defined class hierarchy with a common ancestor (**Object**).
- **Automatic Memory Management:** The runtime includes a garbage collector.
- **Built-in Classes:** Classes such as **Int**, **Bool**, **String**, **I0**, and **Object** are pre-defined.
- **Type Inference in Certain Contexts:** Such as within **let** or **case** expressions.

These features collectively provide a robust environment for exploring fundamental compiler and language design concepts.

System Design

3.1 Architecture of the COOL Compiler

The architecture of the COOL compiler is designed to reflect the modular and phased structure of a modern compiler. It transforms COOL source code into MIPS assembly through a series of well-defined stages. Each stage is implemented as an independent module to facilitate development, testing, and debugging. Note here that symbol table is implemented as stack for this compiler and this idea is discussed in S.S Muchnick's "Advanced Compiler Design and Implementation" [3].

3.1.1 High-Level Components

The compiler consists of the following major components:

- **Lexer (Scanner):** Tokenizes the input stream using regular expressions defined in the COOL language specification.
- **Parser:** Uses context-free grammar (CFG) rules to construct an Abstract Syntax Tree (AST) representing the syntactic structure of the program.
- **Semantic Analyzer:** Performs type checking, scope resolution, inheritance checks, and ensures semantic correctness. It builds and maintains symbol tables and class hierarchies.
- **Code Generators:**
 - **MIPS Code Generator:** Emits MIPS assembly code that can be executed using SPIM or QtSPIM simulators.
- **Runtime Support Library:** Provides implementation for built-in operations such as string manipulation, I/O, and exception handling. It is linked at runtime with the generated code. But only in case of MIPS code generation.

3.1.2 Modular Design

Each compiler phase is separated into different classes or modules:

- `Scanner.l`, `Parser.y` for lexical and syntactic analysis
- `semant.cc` for semantic checking

- `cgen.cc` for MIPS code generation
- `trap.handler` for built-in runtime support functions for COOL language to help in MIPS code generation.

This modularity enables easy extension, such as adding new language features or supporting new backends.

3.1.3 Class Hierarchy and AST Design

The Abstract Syntax Tree (AST) is composed of node classes corresponding to COOL language constructs (e.g., `class_`, `method`, `assign`, `if`, `while`). Each AST node has methods and attributes like `code()` (it basically emits mips, `type_decl` (type of attributes if that ast has some kind of attributes like let expression, class etc have attributes.), `dump_with_types()` (to print ast node attributes and also their static types if applicable) etc which can help us in any phase of compiler to implement that phase successfully.

Class information is stored in the `CgenClassTable`, and scope information is maintained using a hierarchical symbol table structure that supports lexical nesting and inheritance.

3.1.4 Backend Selection and Output

The compiler supports both backends using a compile-time or runtime switch. Depending on the selected backend:

- The MIPS backend emits ‘.s’ files for SPIM simulators.

The generated code is then linked with the runtime support to form a complete executable or interpretable file.

3.2 Phase-wise Design (Lexing to Code Generation)

The COOL compiler is structured as a sequence of distinct compilation phases, each transforming the input program incrementally from high-level COOL source code to low-level target code. Each phase is responsible for specific tasks and is implemented as a separate module to promote modularity and maintainability.

3.2.1 Lexical Analysis (Lexer)

The lexical analyzer, also known as the scanner, reads the raw source code and divides it into a stream of tokens. Tokens represent syntactic categories such as keywords, identifiers, operators, and constants. The scanner for COOL is implemented using `Flex`, a tool for generating scanners from regular expressions.

- Removes whitespace and comments.
- Identifies tokens like `class`, `if`, `while`, identifiers, and literals.
- Returns tokens with line numbers for error reporting.

3.2.2 Syntax Analysis (Parser)

The parser consumes the token stream and constructs an Abstract Syntax Tree (AST) based on the context-free grammar of COOL. It checks the syntactic structure of the program.

- Implemented using **Bison**.
- Provides meaningful error messages for syntax errors.
- Produces a tree structure representing class definitions, expressions, and method declarations.
- Uses `yyerror`, `yyclear` etc functionality to recover from error if any error is encountered during parsing.

3.2.3 Semantic Analysis

Semantic analysis ensures that the program is meaningful according to the language's rules. This includes type checking, scope validation, inheritance correctness, and feature resolution.

- Constructs and traverses the class hierarchy graph.
- Uses a symbol table to track scopes and declarations.
- Ensures expressions have valid types and method calls are correctly formed.
- Reports semantic errors like undefined variables, type mismatches, or cyclic inheritance.

3.2.4 Intermediate Representation (IR) Generation

The AST is traversed to generate an intermediate representation of the program. This abstraction allows backend independence and serves as a clean target for optimizations or transformations.

- Each AST node contains a `code()` method responsible for generating its corresponding output.

3.2.5 Code Generation

The final stage is converting the intermediate representation to target code. The COOL compiler supports both:

- **MIPS Assembly Generation:**
 - Uses registers like `$a0`, `$s0`, and emits standard instructions (`addiu`, `sw`, `lw`, etc.).
 - Output can be tested in SPIM/QtSPIM.

3.2.6 Runtime Integration

The generated code is linked with runtime support functions implemented in MIPS assembly. These include:

- `abort()`: Halts the program.
- `String.length`, `String.substr`, `String.concat`: String operations.
- `I0.out_int`, `I0.out_string`, `I0.in_int`, `I0.in_string`: Input/output etc.

3.2.7 Phase Coordination

All phases are tightly integrated using clear data contracts:

- Tokens $\rightarrow$ Parser
- AST $\rightarrow$ Semantic Analyzer
- Checked AST $\rightarrow$ Code Generator

Each phase prints debug or error output to aid testing, and phase-specific flags allow partial compilation (e.g., stop after parsing or semantic analysis).

3.3 Tools and Technologies Used

3.3.1 Tools and Technologies Used

The development of the COOL compiler involved several tools and technologies that facilitated the implementation of various compiler phases, code generation, and runtime support. These tools were chosen to ensure modularity, efficiency, and compatibility with established compiler design practices.

- **Flex:** A lexical analyzer generator used to implement the scanner. It processes the COOL source code and produces a stream of tokens according to defined regular expressions.
- **Bison:** A parser generator utilized to construct the syntax analyzer. It reads grammar rules of COOL and generates a parser that builds the Abstract Syntax Tree (AST).
- **C++:** The primary programming language used to implement the compiler. C++ was chosen for its performance, object-oriented features, and wide support for compiler construction.
- **MIPS Assembly:** The compiler supports a MIPS backend for generating assembly code executable on SPIM or QtSPIM simulators, providing an educational platform for understanding low-level code generation.
- **GNU Make:** Used as the build automation tool to compile the compiler source code efficiently and manage dependencies.

- **Git:** Version control system employed for source code management and collaboration.
- **Overleaf:** An online LaTeX editor used for writing and compiling the thesis document.
- **SPIM/QtSPIM:** Simulator environments used to execute and test generated MIPS assembly code.

Implementation

4.1 Lexical Analyzer (Scanner)

4.1.1 Introduction

The lexical analysis phase, also known as lexing or scanning, is the first step in the compilation process of the COOL (Classroom Object-Oriented Language) compiler. This phase involves converting a stream of characters from the source code into a stream of meaningful tokens that can be further processed by the parser. In the COOL compiler, this task is performed using a Flex-based scanner defined in the file `cool.flex`.

Flex (Fast Lexical Analyzer Generator) is a tool that generates C code for a scanner, based on rules provided in a specification file. The `cool.flex` file defines patterns using regular expressions and associates them with actions written in C/C++ code.

Please refer to COOL manual it has all details about language [1].

4.1.2 Structure of the Flex File

A Flex file consists of three main sections, separated by ‘%%’:

- **Definitions Section:** Contains C/C++ code to be copied verbatim to the top of the generated scanner file. Includes header files, macro definitions, and global variables.
- **Rules Section:** Contains regular expression patterns and their corresponding C/C++ actions to execute when a pattern matches.
- **User Code Section (optional):** Additional C/C++ functions used by the actions.

4.1.3 Key Components of `cool.flex`

Header and Macros

The Flex file begins with necessary headers and macro definitions:

```
// Includes necessary COOL headers
#include <cool-parse.h>
#include <stringtab.h>
#include <utilities.h>
```

```
#include "tokens.h"
```

```
// Declare external functions
extern int curr_lineno;
extern YYSTYPE cool_yylval;
```

This ensures that the scanner can generate and return tokens that are consistent with the parser's expectations. The 'curr_lineno' tracks the current line number to support precise error reporting.

Pattern Rules

Each rule in the rules section has the form:

```
REGEX_PATTERN { ACTION_CODE }
```

Important tokens includes:

- Keywords (e.g., `class`, `if`, `else`) are matched and mapped to their corresponding token enums (e.g., `CLASS`, `IF`, `ELSE`).
- Identifiers are matched using the regular expression `[a-zA-Z][a-zA-Z0-9]*`.
- Integer constants are matched using `[0-9]+`, which are converted to the token `INT_CONST`.
- String constants are matched using `"([^\\""]|\\\"|\\\\)*)"`, which correctly handles escape sequences.
- Comments (e.g., `-` and `(* ... *)`) and whitespace are skipped or ignored by the lexer.
- Unrecognized characters are handled via an `error()` function.

4.1.4 Token Generation and Semantic Values

When a rule is matched, Flex executes the associated action, which may set 'cool_yylval' to carry semantic information. For example:

```
[a-zA-Z_][a-zA-Z0-9_]* {
    cool_yylval.symbol = idtable.add_string(yytext);
    return OBJECTID;
}
```

This stores the identifier as a 'Symbol' and returns it as an 'OBJECTID'.

4.1.5 Error Handling

Unrecognized characters or malformed inputs are handled using:

```
. {
    cerr << "Lexical error at line " << curr_lineno
    << ": unexpected character '" << yytext << "'" << endl;
}
```

This ensures graceful error reporting and allows the compiler to proceed or terminate with helpful feedback.

4.1.6 Integration with the COOL Compiler

The scanner works with the Bison-based parser generated from ‘cool.y’. Flex generates the function ‘yylex()’ which is invoked by the parser to retrieve the next token. Communication between lexer and parser is achieved via:

- Token values returned by ‘return’ statements.
- Semantic information passed through the ‘YYSTYPE’ union (‘cool_yylval’).
- Line tracking for error messages via ‘curr_lineno’.

4.1.7 Improvements and Extensions

Although the current scanner handles all expected tokens and errors, improvements could include:

- Handling of invisible ASCII characters (e.g., bell, backspace).
- Better handling of unterminated strings and nested comments.
- Enhanced Unicode support for identifiers.

4.1.8 Conclusion

The Flex-based lexical analyzer for COOL is a robust and efficient component that converts source code into a structured stream of tokens. Its modular design, use of regular expressions, and tight integration with the parser make it a foundational piece of the compiler’s front end. Proper understanding and enhancement of this component are crucial for extending or optimizing the COOL compiler.

4.2 Parser Design in COOL

4.2.1 Overview

The COOL (Classroom Object-Oriented Language) parser is a crucial component of the compiler that transforms a stream of lexical tokens into an abstract syntax tree (AST), guided by the grammar rules of the language. Implemented using Bison, the COOL parser systematically checks for syntactic correctness and builds hierarchical structures representing the source program.

4.2.2 Parser Components

The COOL parser is defined using the Bison parser generator, and is responsible for converting lexical tokens into structured tree nodes. It includes:

- Declarations for terminals (tokens) and non-terminals.
- Operator precedence and associativity rules.
- Grammar production rules and associated semantic actions.
- Mechanisms for setting source code locations (line numbers).

4.2.3 Semantic Value Union

To accommodate the various types of nodes in the AST, Bison’s `%union` directive is used to define a type-safe union of semantic values. This includes types like `Program`, `Class_`, `Expression`, and others, each mapping directly to AST nodes in the COOL runtime.

4.2.4 Token Definitions

Tokens are explicitly assigned numerical values to maintain compatibility across Bison versions. They include keywords like `CLASS`, `IF`, `LET`, and `NEW`, as well as special tokens like `INT_CONST`, `STR_CONST`, and `OBJECTID`. These terminals represent the smallest syntactic units produced by the lexer.

4.2.5 Grammar and Productions

The grammar rules of COOL define how tokens combine into syntactic constructs. Each non-terminal has an associated type, and productions are accompanied by semantic actions that construct AST nodes. For example:

```
program : class_list
        { @$ = @1;
          ast_root = program($1); }
```

This rule sets the AST root node to be a `Program` node containing all parsed classes.

4.2.6 Handling Classes and Features

Classes are defined using the `class` rule, supporting both simple and inherited forms:

```
class : CLASS TYPEID '{' feature_list '}' ';'
      | CLASS TYPEID INHERITS TYPEID '{' feature_list '}' ';' ;
```

Features within classes are parsed using:

```
feature : OBJECTID '(' formal_list ')' ':' TYPEID '{' expr '}'
        | OBJECTID ':' TYPEID
        | OBJECTID ':' TYPEID ASSIGN expr
```

The above allow for method and attribute declarations.

4.2.7 Expression Parsing

Expression rules handle various COOL constructs, including dispatch, conditional, loops, blocks, and arithmetic operations. Each rule builds an appropriate AST node. For instance:

```
expr : IF expr THEN expr ELSE expr FI
     { $$ = cond($2, $4, $6); }
```

This constructs a conditional expression node.

4.2.8 LET and CASE Expressions

The LET expression supports variable declarations with optional initializers, and allows chaining of multiple declarations:

```
let_body : OBJECTID ':' TYPEID ASSIGN expr IN expr
        | OBJECTID ':' TYPEID IN expr
        | OBJECTID ':' TYPEID ASSIGN expr ',' let_body
        | OBJECTID ':' TYPEID ',' let_body
```

Similarly, CASE expressions are implemented with rules for case lists and branches:

```
case_list : case_
          | case_list case_
```

4.2.9 Block and Dispatch Handling

Blocks are parsed using semicolon-separated expressions:

```
expr : '{' expr_list_semicolon '}'
```

Dispatch is parsed in both static and dynamic forms:

```
expr : expr '@' TYPEID '.' OBJECTID '(' expr_list_comma ')'
      | expr '.' OBJECTID '(' expr_list_comma ')'
```

These enable method calls within class hierarchies.

4.2.10 Operator Precedence

Operator precedence is declared to resolve ambiguities in parsing expressions. For instance:

```
%right ASSIGN
%left NOT
%nonassoc LE '<' '='
%left '+' '-'
%left '*' '/'
```

This ensures correct parsing of compound expressions like `a + b * c`.

4.2.11 Location Tracking and Line Numbers

To provide accurate error messages and AST node annotations, the parser utilizes Bison's location tracking:

```
#define SET_NODELOC(Current) node_lineno = Current;
```

The `YYLTYPE` is defined as `int` and tracks line numbers using `curr_lineno`, which is updated by the lexer.

4.2.12 Error Handling

Parse errors invoke the `yyerror` function:

```
void yyerror(char *s)
{
    cerr << "\"" << curr_filename << "\", line " << curr_lineno << ": "
        << s << " at or near ";
    print_cool_token(yychar);
    cerr << endl;
    omerrs++;

    if(omerrs > 50) exit(1);
}
```

This prints the current line number, the erroneous token, and halts after 50 errors.

4.2.13 Summary

The COOL parser systematically converts input tokens into a well-structured AST, enabling subsequent phases like semantic analysis and code generation. Its design emphasizes clarity, maintainability, and precise error reporting, aligning with the educational goals of the COOL language.

4.3 Semantic Analyzer

4.4 Introduction

Work of semantic analysis is to verify that yes this code belong to the language for which the compiler is being written. Semantic Analysis module takes input the syntax tree build by Bison and it generates AST(Abstract Syntax Tree). AST simplifies codegenration in many aspects for example it simplifies Operational Rules of language etc. The semantic analyzer performs type checking, inheritance validation, and scope management to ensure program correctness before code generation. The semantic analysis phase is crucial for ensuring type safety and proper program structure. This analysis examines the implementation of a semantic analyzer that validates class hierarchies, method signatures, and expression types while maintaining symbol tables and type environments throughout the compilation process. Type Checking rules for COOL and other important stuff is discussed about semantic analysis of COOL language is in COOL manual [1].

4.5 Core Data Structures

4.5.1 Symbol Management

The semantic analyzer employs several key data structures for efficient symbol management. A global class mapping(`std::map<Symbol, Class_> class_map`) maintains associations between class names and their corresponding class objects, enabling quick class

lookup operations. Additionally, a method environment uses paired identifiers consisting of class names and method names to facilitate method resolution across the inheritance hierarchy. Basically this method environment implements M that we have seen in Type Checking Rules in COOL manual.

4.5.2 Type Environment

The type environment serves as the central context for semantic analysis, maintaining two critical components: the current class context representing the class being analyzed, and an object environment that functions as a symbol table for variable bindings and scope management.

4.6 Class Hierarchy Validation

4.6.1 Basic Class Installation

The semantic analyzer begins by installing predefined basic classes that form the foundation of the COOL type system. The `Object` class serves as the root of the inheritance hierarchy, providing fundamental methods for program termination, type identification, and object copying. The `I0` class extends `Object` with input/output capabilities, while primitive types `Int`, `Bool`, and `String` provide basic data manipulation with their respective operations.

4.6.2 Inheritance Validation

The class table constructor performs comprehensive inheritance validation through multiple phases. Initially, it checks for class redefinitions to prevent naming conflicts. Subsequently, it validates inheritance chains by ensuring parent classes exist and are properly defined. The system prevents inheritance from basic primitive types and detects circular inheritance patterns that would create undefined behavior.

4.7 Method Environment Construction

4.7.1 Global Method Environment[M]

The construction of a global method environment(`method_class *lookup_method(Symbol class_name, Symbol method_name)`) involves traversing all defined classes and cataloging their methods. This process creates a comprehensive mapping that associates class-method pairs with their corresponding method implementations, enabling efficient method lookup during type checking. Basically this are the M component respectively that we have seen during COOL Typechecking Rules in COOL manual.

4.7.2 Method Resolution

Method resolution follows the inheritance hierarchy systematically, beginning with the target class and ascending through parent classes until the desired method is found. This approach ensures proper method overriding semantics and maintains consistency

with object-oriented inheritance principles. Algorithm for is similar to dispatch table construction.

4.8 Type Checking Framework

4.8.1 Subtyping Relations

The subtyping system(`bool is_subclass(Symbol sub, Symbol super, type_env & tenv)`) implements the fundamental "is-a" relationship central to object-oriented programming. The subtype checking algorithm traverses the inheritance hierarchy to determine whether one type can be safely substituted for another, handling special cases such as `SELF_TYPE` which represents the current class context.

Algorithm 1: Subtype Checking Algorithm

```

1 Function isSubtype( $T_1, T_2$ ):
2   if  $T_1 = T_2$  then
3     return true;
4   else if  $T_1 = SELF\_TYPE$  then
5     return  $T_2 = SELF\_TYPE$ ;
6   else if  $T_2 = SELF\_TYPE$  then
7     return false;
8   else
9      $C_1 \leftarrow \text{getClass}(T_1)$ ;
10     $C_2 \leftarrow \text{getClass}(T_2)$ ;
11    return isAncestor( $C_1, C_2$ );

```

4.8.2 Type Join Operations

Type join operations(`Symbol cls_join(Symbol a, Symbol b, type_env & tenv)`) compute the least upper bound of two types in the inheritance hierarchy, which is essential for determining result types in conditional expressions and other constructs where multiple types must be unified. The algorithm systematically traverses upward through the class hierarchy until a common ancestor is found.

4.9 Expression Type Checking

4.9.1 Introduction

Type checking is done according to Type Checking Rules given in COOL manual. i have only explained and given codes for few type checking rules, as there are many constructs so many type checking are there in COOL manual. To know deeply about type checking please refer github code base of this project and COOL manual. Type checking here is done using DFS in bottom up manner means first we do type checking children nodes in half filled AST then we type checking for its parent.

4.9.2 Assignment Expressions

Assignment expression validation ensures type safety by verifying that assigned values conform to the declared types of target variables. The system prevents assignments to the special 'self' identifier as self can be used as return statement of a function not as type use instead SELF_TYPE and validates that assigned expressions are subtypes of the variable's declared type. Code for this scenario can be found in listing 4.1.

Listing 4.1: Type checking for assignment in COOL semantic analyzer

```
Symbol assign_class::typecheck(type_env &tenv) {
    type = Object;

    if (name == self) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Cannot assign to 'self'." << std::endl;
        return type;
    }

    Symbol *t = tenv.o.lookup(name);

    if (!t) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Assignment to undeclared variable " << name << "."
            << std::endl;
        return type;
    }

    Symbol t_ = expr->typecheck(tenv);

    if (is_subclass(t_, *t, tenv) == false) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Type " << t_ << " of assigned expression does not
            << conform to "
            << "declared type " << *t << " of identifier " << name
            << std::endl;
        return type;
    }

    type = t_;
    return type;
}
```

4.9.3 Method Dispatch

Method dispatch validation encompasses both static and dynamic dispatch scenarios. The system verifies method existence, checks argument types against formal parameters, and determines appropriate return types. Special handling is required for SELF_TYPE returns, which must be resolved to the actual receiver type. These are two types of method dispatch static and dynamic, codes are given for these two scenarios in listing 4.2 and listing 4.3 respectively.

Listing 4.2: Static dispatch type checking in COOL

```

Symbol static_dispatch_class::typecheck(type_env &tenv) {
    Symbol t0 = expr->typecheck(tenv);
    Symbol t = type_name;

    if (!is_subclass(t0, t, tenv)) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Expression type " << t0 << " does not conform to declared
            ↪ static "
            "dispatch type " << t << "." << std::endl;
    }

    method_class *method = lookup_method(t, name);
    if (!method) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Dispatch to undefined method " << name << "." << std::endl
            ↪ ;
        type = Object;
        return type;
    }

    Formals formals = method->get_formals();
    bool formals_are_less = false;
    int i;

    for (i = actual->first(); actual->more(i); i = actual->next(i)) {
        Symbol t_actual = actual->nth(i)->typecheck(tenv);

        if (formals->more(i)) {
            Formal f = formals->nth(i);
            Symbol t_formal = f->get_type_decl();

            if (!is_subclass(t_actual, t_formal, tenv)) {
                classtable->semant_error(tenv.c->get_filename(), this)
                ↪ <<
                    "In call of method " << name << ", type " <<
                    ↪ t_actual <<
                    " of parameter " << f->get_name() << " does not
                    ↪ conform to "
                    "declared type " << t_formal << "." << std::endl;
            }
        } else {
            formals_are_less = true;
        }
    }

    if (formals_are_less || formals->more(i)) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Method " << name << " called with wrong number of
            ↪ arguments."
            << std::endl;
    }

    type = method->get_return_type();
    if (type == SELF_TYPE) {
        type = t0;
    }
}

```

```

    return type;
}

```

Listing 4.3: Dynamic dispatch type checking in COOL

```

Symbol dispatch_class::typecheck(type_env &tenv) {
    Symbol t0 = expr->typecheck(tenv);
    Symbol t0_ = t0;

    if (t0_ == SELF_TYPE) {
        t0_ = tenv.c->get_name();
    }

    method_class *method = lookup_method(t0_, name);
    if (!method) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Dispatch to undefined method " << name << "." << std::endl
            ↪ ;
        type = Object;
        return type;
    }

    Formals formals = method->get_formals();
    bool formals_are_less = false;
    int i;

    for (i = actual->first(); actual->more(i); i = actual->next(i)) {
        Symbol t_actual = actual->nth(i)->typecheck(tenv);

        if (formals->more(i)) {
            Formal f = formals->nth(i);
            Symbol t_formal = f->get_type_decl();

            if (!is_subclass(t_actual, t_formal, tenv)) {
                classtable->semant_error(tenv.c->get_filename(), this)
                    ↪ <<
                    "In call of method " << name << ", type " <<
                    ↪ t_actual <<
                    " of parameter " << f->get_name() << " does not
                    ↪ conform to "
                    "declared type " << t_formal << "." << std::endl;
            }
        } else {
            formals_are_less = true;
        }
    }

    if (formals_are_less || formals->more(i)) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Method " << name << " called with wrong number of
            ↪ arguments."
            << std::endl;
    }

    type = method->get_return_type();
    if (type == SELF_TYPE) {
        type = t0;
    }
}

```

```

    return type;
}

```

4.9.4 Conditional Expressions

Conditional expression type checking requires Boolean predicates and computes result types by joining the types of then and else branches. This ensures type safety while allowing maximum flexibility in conditional logic. Code for this type checking given in listing 4.4.

Listing 4.4: Conditional expression type checking in COOL

```

Symbol cond_class::typecheck(type_env &tenv) {
    Symbol t1 = pred->typecheck(tenv);
    Symbol t2 = then_exp->typecheck(tenv);
    Symbol t3 = else_exp->typecheck(tenv);

    if (t1 != Bool) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Predicate of 'if' does not have type Bool." << std::endl;
    }

    type = cls_join(t2, t3, tenv);
    return type;
}

```

4.9.5 Case Expressions

Case expression validation includes checking for duplicate branch types and ensuring proper scope management for pattern variables. The result type is computed as the join of all branch types, providing type safety across all possible execution paths. Dynamic type which is more accurate can only be known at run time for this construct. Code for this type checking given in listing 4.5.

Listing 4.5: Type checking for case expressions in COOL

```

Symbol typcase_class::typecheck(type_env &tenv) {
    Symbol t0 = expr->typecheck(tenv);

    std::vector<Symbol> used;

    for (int i = cases->first(); cases->more(i); i = cases->next(i)) {
        Symbol prev_type = type;

        Case c = cases->nth(i);
        Symbol type_decl = c->get_type_decl();

        if (std::find(used.begin(), used.end(), type_decl) == used.end()
            ↪ ()) {
            used.push_back(type_decl);
        } else {
            classtable->semant_error(tenv.c->get_filename(), this) <<
                "Duplicate branch " << type_decl << " in case statement
                ↪ ."
                << std::endl;
        }
    }
}

```

```

        type = Object;
        return type;
    }

    tenv.o.enterscope();
    tenv.o.addid(c->get_name(), new Symbol(type_decl));
    type = c->get_expr()->typecheck(tenv);

    if (i > 0) {
        type = cls_join(type, prev_type, tenv);
    }

    tenv.o.exitscope();
}

return type;
}

```

4.10 Scope Management

4.10.1 Object Environment Initialization

The object environment initialization process creates comprehensive symbol tables by incorporating both inherited attributes from parent classes and locally declared attributes. The system performs redefinition checking to prevent attribute name conflicts while maintaining proper inheritance semantics.

4.10.2 Scope Handling

Scope management utilizes a stack-based approach where new scopes are entered for method bodies, let expressions, and case branches. This ensures proper variable visibility and prevents name conflicts between different lexical scopes.

4.11 Method Validation

Method validation encompasses multiple aspects of method correctness. Parameter validation ensures that formal parameters have valid types and unique names, while preventing the use of `'self'` as a parameter name. Return type validation verifies that method bodies produce values conforming to declared return types. For overridden methods, the system enforces signature compatibility by checking parameter types and counts against parent class method definitions.

4.12 Attribute Management

Attribute validation ensures that class attributes have appropriate types and initializer expressions. The system prevents the use of `'self'` as an attribute name and validates that initialization expressions, when present, conform to declared attribute types. Inheritance relationships are respected by checking for attribute redefinitions across the class hierarchy.

4.13 Special Type Handling

4.13.1 SELF_TYPE Semantics

`SELF_TYPE` represents a special polymorphic type that refers to the class of the current object. The semantic analyzer handles `SELF_TYPE` carefully in various contexts, ensuring proper type resolution in method returns, parameter types, and subtyping relationships.

4.13.2 Primitive Type Operations

Primitive types `Int`, `Bool`, and `String` have special semantic rules for their operations. Arithmetic operations require integer operands, comparison operations have specific type requirements, and string operations follow their own semantic rules. Codes for some of basic operation are given below.

4.13.3 Arithmetic Operation

Listing 4.6: Type checking for plus operation

```
Symbol plus_class::typecheck(type_env &tenv) {
    Symbol t_e1 = e1->typecheck(tenv);
    Symbol t_e2 = e2->typecheck(tenv);

    if (t_e1 != Int || t_e2 != Int) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "non-Int arguments: " << t_e1 << " + " << t_e2 << std::endl
        ↪ ;
        type = Object;
    } else {
        type = Int;
    }

    return type;
}
```

4.13.4 Boolean Operation

Listing 4.7: Type checking for complement operation

```
Symbol comp_class::typecheck(type_env &tenv) {
    type = e1->typecheck(tenv);

    if (type != Bool) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "Argument of 'not' has type " << type << " instead of Bool.
        ↪ "
        << std::endl;
    }

    type = Bool;
    return type;
}
```

4.14 Constant Expressions

Listing 4.8: Type checking for constant expressions

```
Symbol int_const_class::typecheck(type_env &tenv) {
    type = Int;
    return type;
}

Symbol bool_const_class::typecheck(type_env &tenv) {
    type = Bool;
    return type;
}

Symbol string_const_class::typecheck(type_env &tenv) {
    type = Str;
    return type;
}
```

4.15 Special Expressions

4.15.1 Object Creation

Listing 4.9: Type checking for new expression

```
Symbol new__class::typecheck(type_env &tenv) {
    Symbol t = type_name;

    if (!cls_is_defined(t)) {
        classtable->semant_error(tenv.c->get_filename(), this) <<
            "'new' used with undefined class " << t << "." << std::endl
            ↪ ;
        type = Object;
    } else {
        type = t;
    }

    return type;
}
```

4.15.2 Void Check

Listing 4.10: Type checking for isvoid expression

```
Symbol isvoid_class::typecheck(type_env &tenv) {
    e1->typecheck(tenv);
    type = Bool;
    return type;
}
```

4.15.3 Empty Expression

Listing 4.11: Type checking for no expression

```
Symbol no_expr_class::typecheck(type_env &tenv) {
    type = No_type;
    return type;
}
```

4.16 Error Reporting

The semantic analyzer provides comprehensive error reporting with precise location information including filenames and line numbers. Error messages are descriptive and indicate the specific nature of semantic violations, enabling programmers to identify and correct issues efficiently.

4.17 Analysis Phases

4.17.1 Class Table Construction

The initial phase constructs the class table by processing all class declarations, validating inheritance relationships, and ensuring the presence of required classes such as `Main`. This phase establishes the foundation for all subsequent semantic analysis.

4.17.2 Method Environment Building

The second phase constructs the global method environment by cataloging all method definitions across all classes. This creates the necessary infrastructure for method lookup and validation.

4.17.3 Type Checking Phase

The final phase performs comprehensive type checking by analyzing each class individually, validating all expressions, methods, and attributes within their appropriate type environments.

4.18 Integration with Compilation Pipeline

The semantic analyzer integrates seamlessly with other compilation phases by decorating the abstract syntax tree with type information and providing error reporting that halts compilation when semantic errors are detected. This ensures that only semantically correct programs proceed to code generation.

4.19 Code Generation (MIPS)

4.19.1 Introduction

This thesis explains the code generation process for the COOL (Classroom Object-Oriented Language) programming language, targeting MIPS assembly. The code generator takes COOL's abstract syntax tree (AST) as input and produces MIPS assembly code that can be executed on a MIPS processor or simulator. Operational Semantics, runtime error for COOL and other important stuff is discussed about code generation of COOL language is in COOL manual [1].

4.19.2 Overall Architecture

The code generator consists of several key components:

Explanation of Architecture

The `CodeGenerator` module is responsible for translating the abstract syntax tree (AST) of a COOL program into MIPS assembly code. To achieve this, it constructs several helper classes and data structures that facilitate the correct and efficient generation of

code for each language construct. These components collectively ensure that the context necessary for each AST node is available during code generation, thereby enabling the production of valid and semantically accurate MIPS instructions, in other words we are gluing up syntax or AST nodes correctly.

The architecture comprises the following core components:

- **Environment:** This class acts as a scoping mechanism for the COOL language. It maintains the state of variables within the program, prioritizing variables in nested scopes using a stack-based logic. It also provides utility functions for managing variable lifetimes and lookup operations, ensuring proper handling of name collisions and scope resolution.
- **ClassTable:** This structure maintains the class hierarchy by tracking all inheritance relationships using a vector-based representation. It plays a crucial role in organizing class dependencies and facilitating inheritance-aware code generation.
- **Dispatch Table:** This component generates dispatch tables for each class. These tables store method pointers and support dynamic method dispatch during runtime. Algorithm for dispatch table generation can be found in subsection 4.7.2.
- **Methods:** This helper class is responsible for generating code for all methods across the class hierarchy. It ensures that method definitions are correctly compiled and inserted into the appropriate locations in the MIPS output. And detailed algorithm can be found in subsection 4.7.4.
- **Object Initialization:** This component handles the generation of MIPS code required to initialize class objects at runtime. It sets up memory structures, populates attribute fields, and prepares objects for execution. The detailed algorithm for this process is discussed in subsection 4.7..
- **Class Tag And ClassMap:** Each class is assigned a unique tag (an integer ID starting from 1 which is used in dispatch tables, type checks, and object creation. Additionally, the ClassMap maintains mappings from class names to their respective AST nodes and class tags, which uniquely identify each class.
- **Install Class protObj:** This function generates MIPS data sections in `.word` format to configure prototype objects. It enables runtime object creation using the `Object.copy` function. The prototype object includes fields for class tag, object size, dispatch table pointer, inherited attributes, local attributes, and a garbage collection tag, which indicates whether the object is managed by the garbage collector. And detailed algorithm can be found in subsection 4.7.3.
- **Install Class:** This function installs all fundamental classes required by the COOL language, such as `Object`, `IO`, `String`, `Int`, and `Bool`. It also includes the setup of associated class methods and properties to ensure completeness of the runtime environment.
- **Object Tables:** This final component handles the generation of all essential runtime objects using the `StringTable`, `IntTable`, and `BoolTable`. These tables prevent redundancy by sharing instances of objects with the same constant values. First, the configuration for each object is emitted, followed by their instantiation during runtime.

Collectively, these components form the backbone of the code generation architecture, enabling a seamless transformation from COOL's high-level constructs to low-level MIPS assembly instructions.

Dispatch Table Construction Algorithm

Dispatch tables are central to supporting dynamic method dispatch in object-oriented programming languages like COOL. Each class maintains a dispatch table, which maps method names to their respective implementation addresses. To ensure proper inheritance and method overriding (like in c++ virtual function), methods from all ancestor classes must be collected, with overridden methods replaced by the most specific implementation.

Algorithm: Recursive Method Collection

The following algorithm recursively traverses the inheritance hierarchy of a given class, collecting all methods from its ancestors and the class itself. If a method is overridden in a child class, it replaces the method from the parent.

Algorithm 2: Recursive Method Collection for Dispatch Table

Input: Class `cls`, method list `methods`

Output: All inherited and overridden methods collected for dispatch table

```

1 if cls.name  $\neq$  "Object" then
2   | get_methods_recursively(parent(cls), methods);
3 features  $\leftarrow$  cls.get_features();
4 foreach f in features do
5   | if f is not a method then
6   |   | continue;
7   | method  $\leftarrow$  (Method) f ;                               // Cast feature to method
8   | name_already_exists  $\leftarrow$  false;
9   | foreach (owner, m) in methods do
10  |   | if m.name = method.name then
11  |   |   | name_already_exists  $\leftarrow$  true;
12  |   |   | owner  $\leftarrow$  cls ;                               // Override inherited method
13  |   | if not name_already_exists then
14  |   |   | methods.push_back((cls, method));

```

Explanation

This recursive algorithm traverses up the inheritance tree starting from the given class until it reaches the base class `Object`. At each level, it inspects the features of the class. If the feature is a method, it checks if a method with the same name already exists in the collected list:

- If it does, the method is overridden by updating the owner class in the existing pair.
- If it does not, the method is appended to the list.

This ensures that each method in the final dispatch table reflects the most derived implementation available in the hierarchy.

Algorithm: Dispatch Table Generation

Once all methods have been collected, the dispatch tables for all classes are generated using the following algorithm:

Algorithm 3: Dispatch Table Generation

Input: List of classes `cls_ordered`

Output: Dispatch tables generated for all classes

```

1 foreach cls in cls_ordered do
2   Emit label: cls.name_dispTab::;
3   get_methods_recursively(cls, cls.all_methods);
4   foreach (owner, method) in cls.all_methods do
5     | Emit word: owner.name.method.name;
6   if cls.name = "Main" then
7     | Emit word: -1;
```

Explanation

This procedure iterates over all classes in the topological order of inheritance (ensured by `cls_ordered`). For each class:

- A unique dispatch table label is emitted.
- Methods are collected recursively from parent classes.
- Each method entry is emitted in the dispatch table using the format `Class.Method`.
- A sentinel value `-1` is added at the end of the `Main` class dispatch table, if required.

Significance

This mechanism ensures that:

- Each class's dispatch table includes inherited and overridden methods in the correct order.
- Method calls using dynamic dispatch (e.g., `obj.method()`) can be resolved efficiently at runtime via table lookups.
- The overriding semantics of COOL are respected and implemented correctly in the generated MIPS assembly.

4.19.3 Prototype Object Installation

In the code generation phase of a compiler, prototype objects are emitted into the `.data` section of the target code. These objects represent the layout of class instances at runtime and are essential for object instantiation via copying. The following algorithm emits a prototype object for a given class.

Explanation. This algorithm emits a prototype object in the assembly's `.data` section, which is used during object instantiation. Each class is assigned a unique `class_tag` and a fixed layout size. Depending on whether the class is a built-in type (`Int`, `Bool`, `String`, `Void`) or user-defined, the algorithm handles their default values differently. For built-in types, the layout is known and minimal; for user-defined types, each attribute's default value is determined by its declared type. Finally, a `.word -1` is emitted to mark the end of the prototype object's layout, often used by the garbage collector or for debugging.

Algorithm 4: Prototype Object Installation

Input: AST node *nd*, class table *ct*

Output: Emit prototype object for class *nd* into data section

```
1 class_name ← nd.get_name();
2 class_tag ← get_class_tag(class_name);
3 attr_count ← number of attributes in nd;
4 object_size ← DEFAULT_OBJFIELDS + attr_count;
5 Emit label class_name_protObj;;
6 Emit WORD class_tag;
7 if class_name = "Int" or class_name = "Bool" then
8   object_size ← 4;
9   Emit WORD object_size;
10  Emit WORD dispatch table reference for class_name;
11  Emit WORD reference to constant 0;
12 else if class_name = "String" then
13   object_size ← 5;
14   Emit WORD object_size;
15   Emit WORD dispatch table reference for class_name;
16   Emit WORD int_const0;
17   Emit WORD 0;
18 else if class_name = "Void" then
19   object_size ← 4;
20   Emit WORD object_size;
21   Emit WORD dispatch table reference for class_name;
22   Emit WORD 0 ;                               // null
23 else
24   object_size ← 3 + attr_count;
25   Emit WORD object_size;
26   Emit WORD dispatch table reference for class_name;
27   foreach attribute attr in nd.all_attrs do
28     attr_type ← attr.type_decl;
29     if attr_type = "Int" then
30       Emit WORD int_const_maxIndexint;
31     else if attr_type = "Bool" then
32       Emit WORD bool_const0;
33     else if attr_type = "String" then
34       Emit WORD str_const(numstr_const - 1);
35     else
36       Emit WORD 0;
37 Emit .word -1 ;                               // Garbage Collector terminator
```

4.19.4 Method Code Generation: `install_methods`

Explanation

The `install_methods` is responsible for emitting MIPS assembly code for method definitions in each class. The method operates as follows:

- **Prologue Emission:** The function first stores the caller's state, including the frame pointer (`$fp`), return address (`$ra`), and the current object register (`$s0`), onto the stack. It then adjusts the frame pointer and assigns the current object (`$a0`) to `$s0` for use within the method.
- **Inherited Attribute Initialization:** Starting from the current class node, the function traverses up the inheritance chain to the `Object` class. It collects attributes from parent classes and stores initialized attributes into the local environment. This ensures that inherited attributes are available when generating code for the method body.
- **Formal Parameter Setup:** All formal parameters declared in the method are added to the environment. This step is crucial to allow correct variable resolution in the expression code generation phase.
- **Method Body Code Generation:** The body of the method is compiled into MIPS assembly using the current environment, which includes inherited attributes and method parameters.
- **Epilogue Emission:** After the method body code has been generated, the function restores the stack by loading saved values of `$fp`, `$s0`, and `$ra`, adjusting the stack pointer, and finally emitting the return instruction.

Algorithm 5: Method Installation and Code Generation

Input: Class node *nd*, class table *ct*, method *method*

Output: Emits MIPS assembly code for the given method

```
1 Emit stack frame prologue;;
2   Store $fp, $s0, and $ra on stack;
3   Update $fp, set $s0  $\leftarrow$  $a0;
4 Initialize environment env;
5 grand_name  $\leftarrow$  nd;
6 while grand_name  $\neq$  "Object" do
7   | Add grand_name to grand vector;
8   | grand_name  $\leftarrow$  parent class from classMap;
9 foreach class c in reverse of grand do
10  | foreach feature f in c do
11  |   | if f is an attribute with initializer then
12  |   |   | Add attribute to env;
13 foreach formal parameter arg in method do
14  | Add arg to env;
15 Generate code for method body using method->expr->code(env);
16 Clear method arguments from env;
17 Emit stack frame epilogue;;
18   Restore $fp, $s0, and $ra;
19   Adjust stack pointer and emit return;
```

4.19.5 Object Initializer Method

Explanation

The `object_initial` method generates MIPS assembly code to initialize an object instance of a given class in the COOL language. This method is responsible for:

- Emitting the function prologue.
- Calling the parent class initializer if the current class is not `Object`.
- Traversing the class hierarchy from child to `Object` to handle attribute initializations.
- Adding initialized attributes to the environment for later access.
- Emitting code to initialize attributes with user-defined expressions.
- Emitting the function epilogue and returning control to the caller.

To avoid reinitializing inherited attributes multiple times, the method performs a reverse traversal of the inheritance chain and tracks attribute positions using the environment object `env`.

Algorithm 6: Object Initialization for Class *cls*

Input: Class *cls*, Class Table *ct*

Output: Emits MIPS code to initialize *cls* and its attributes

```
1 class_name ← cls->get_name();
2 parent_name ← cls->get_parent();
3 Emit label: class_name_init::;
4 Emit stack frame prologue;;
5   Store $fp, $s0, $ra on stack;
6   Update $fp, set $s0 ← $a0;
7 if class_name ≠ "Object" then
8   | Emit jump to parent_name_init();
9 Initialize grand list with class hierarchy up to Object;
10 grand_name ← class_name;
11 while grand_name ≠ "Object" do
12   | Append grand_name to grand;
13   | grand_name ← classMap[grand_name->get_parent()];
14 foreach class c in reverse of grand do
15   | foreach feature f in c do
16     | if f is attribute with initializer then
17       | | Add attribute to environment env;
18 attrMap[class] ← attrMap[class] + attrMap[parent];
19 foreach attribute a in cls->features() do
20   | if attribute has non-empty initializer then
21     | Generate code for a->init;
22     | offset ← 3 + env.get_cls_attr_pos(a);
23     | Store result in object at $s0 + offset;
24 Emit function epilogue;;
25   Move $a0 ← $s0;
26   Restore $fp, $s0, $ra;
27 Adjust stack pointer and return;
```

4.19.6 Class Representation

```
std::vector<Symbol> class_tag; // Ordered list of class names
std::unordered_map<Symbol, int> attrMap; // Maps classes to attribute counts
std::vector<Class_> cls_ordered; // Classes in inheritance order
std::unordered_map<Symbol, Class_> classMap; // Maps class names to AST nodes
```

4.19.7 Environment Tracking

Purpose and Role

The `Environment` class is used during code generation to keep track of the current context in which expressions are evaluated and code is emitted. It acts as a *symbolic environment manager* that holds:

- The current class context (`cls`)
- A list of class-level attributes
- A list of method/formal arguments
- A symbolic stack for ‘let’ variables and other temporaries

This design allows the code generator to accurately locate where a variable resides: whether it is a class attribute, a method argument, or a local ‘let’ binding.

Environment Class Implementation

Listing 4.12: Environment Class Definition

```
class Environment {
    Class_ cls;
    std::vector<attr_class *> cls_attrs;
    std::vector<Formal> mth_args;
    std::vector<Symbol> stack_symbols;

public:
    Class_ get_cls() { return cls; }
    void set_cls(Class_ cls) { this->cls = cls; }

    int get_cls_attrs_size() { return cls_attrs.size(); }
    int get_mth_args_size() { return mth_args.size(); }

    void add_cls_attr(attr_class *attr) { cls_attrs.push_back(attr); }
    void add_mth_arg(Formal formal) { mth_args.push_back(formal); }
    void clear_mth_args() { mth_args.clear(); }

    void push_stack_symbol(Symbol name) { stack_symbols.push_back(name); }
    void pop_stack_symbol() { stack_symbols.pop_back(); }

    int get_let_var_pos_rev(Symbol name) {
        for (int i = stack_symbols.size() - 1; i >= 0; i--) {
            if (stack_symbols[i] == name)
                return stack_symbols.size() - 1 - i;
        }
    }
}
```

```

    }
    return -1;
}

int get_arg_pos(Symbol name) {
    for (int i = 0; i < (int) mth_args.size(); i++) {
        if (mth_args[i]->get_name() == name) return i;
    }
    return -1;
}

int get_cls_attr_pos(Symbol name) {
    for (int i = 0; i < (int) cls_attrs.size(); i++) {
        if (cls_attrs[i]->get_name() == name) return i;
    }
    return -1;
}
};

extern Environment env;

```

Algorithm 7: Environment Symbol Resolution

Input: A variable name *name*

Output: Return position and scope of the variable (stack, method args, or class attr)

```

1 if name is found in stack symbols then
2   | return position in stack;
  | // Scope is let/local
3 else if name is found in method arguments then
4   | return position in method arguments;
  | // Scope is argument
5 else if name is found in class attributes then
6   | return position in class attributes;
  | // Scope is class attribute
7 else
8   | return-1;
  | // Variable is undefined

```

4.19.8 Explanation of Components

- **Class Context:** The ‘cls’ field helps associate generated code with the current class.
- **Attributes:** The ‘cls_attrs’ vector stores attributes, which are useful for attribute offset calculation in the object layout.
- **Method Arguments:** Stored in ‘mth_args’, these are checked before attributes or locals during resolution.
- **Stack Symbols:** Implements a reverse-search stack for managing nested ‘let’ expressions or scoped variables.

- **Reverse Lookup:** The function ‘get_let_var_pos_rev’ is essential to simulate stack-like lookup, enabling shadowing detection.

4.20 Code Generation Process

4.20.1 Initialization

The code generation begins with `program_class::cgen()`, which:

- Initializes predefined symbols (Object, Int, Bool, etc.)
- Creates the class table
- Generates global data and text segments

4.20.2 Class Table Construction

The `CgenClassTable` constructor:

- Installs basic classes (Object, IO, Int, Bool, String)
- Processes user-defined classes
- Builds the inheritance tree
- Generates code for class name table, object table, dispatch tables, prototype objects, initialization methods, and class methods

e.g I) Dispatch Tables

```
void CgenClassTable::code_dispatch_tables() {
    for (auto clsy : cls_ordered) {
        str << clsy->get_name() << DISPTAB_SUFFIX << LABEL;
        get_methods_recursively(clsy, clsy->all_methods);
        for (auto method : clsy->all_methods) {
            str << WORD << method.first->get_name() << "."
                << method.second->get_name() << endl;
        }
    }
}
```

e.g II) Prototype Objects

```
void CgenClassTable::install_classprot(Class_ nd, CgenClassTableP ct) {
    Symbol class_name = nd->get_name();
    int class_tag = get_class_tag(class_name);
    int attr_count = nd->all_attrs.size();
    int object_size = DEFAULT_OBJFIELDS + attr_count;

    emit_protobj_ref(class_name, str);
}
```

```

str << LABEL << WORD << class_tag << endl << WORD << object_size << endl;
str << WORD; emit_disptable_ref(class_name, str); str << endl;

for (auto attr : nd->all_attrs) {
    Symbol attr_type = attr->type_decl;
    if (attr_type == Int) {
        str << WORD << "int_const0" << endl;
    } else if (attr_type == Bool) {
        str << WORD << "bool_const0" << endl;
    } else if (attr_type == Str) {
        str << WORD << "str_const0" << endl;
    } else {
        str << WORD << "0" << endl;
    }
}
}

```

4.20.3 e.g III) Object Initialization

```

void CgenClassTable::object_initial(Class_ cls, CgenClassTableP ct) {
    // Prologue
    // ... setup stack frame ...

    // Call parent initializer
    if (class_name != Object) {
        emit_jal(parent_name->get_string() + "_init", str);
    }

    // Initialize attributes
    for (auto attr : cls->all_attrs) {
        if (!dynamic_cast<no_expr_class*>(attr->init)) {
            attr->init->code(str, env);
            int offset = 3 + env.get_cls_attr_pos(attr->get_name());
            emit_store(ACC, offset, S0, str);
        }
    }

    // Epilogue
    // ... restore stack frame ...
}

```

e.g IV) Method Generation

```

void CgenClassTable::install_methods(Class_ nd, method_class* method) {
    // Prologue
    // ... save registers ...

    // Process arguments

```

```

    int arg_count = 0;
    for (int i = method->formals->first(); method->formals->more(i);
        i = method->formals->next(i)) {
        env.add_mth_arg(method->formals->nth(i));
        arg_count++;
    }

    // Generate body
    method->expr->code(str, env);

    // Epilogue
    // ... restore and return ...
}

```

4.21 Expression Code Generation

e.g I) Arithmetic Operations

```

void plus_class::code(ostream &s, Environment &env) {
    e1->code(s, env);
    emit_push(ACC, s);
    e2->code(s, env);
    emit_jal("Object.copy", s);
    emit_addiu(SP, SP, 4, s);
    emit_load(T1, 0, SP, s);
    emit_move(T2, ACC, s);
    emit_fetch_int(T1, T1, s);
    emit_fetch_int(T2, T2, s);
    emit_add(T3, T1, T2, s);
    emit_store(T3, 3, ACC, s);
}

```

e.g II) Control Structures

```

void cond_class::code(ostream &s, Environment &env) {
    pred->code(s, env);
    emit_fetch_int(T1, ACC, s);
    int label_false = label_num++;
    int label_end = label_num++;
    emit_beq(T1, ZERO, label_false, s);
    then_exp->code(s, env);
    emit_branch(label_end, s);
    emit_label_def(label_false, s);
    else_exp->code(s, env);
    emit_label_def(label_end, s);
}

```

e.g III) Object Dispatch

```
void dispatch_class::code(ostream &s, Environment &env) {
    // Push arguments
    // ... push and eval ...

    // Evaluate receiver
    expr->code(s, env);
    emit_bne(ACC, ZERO, label_num, s);
    // Error handling
    emit_label_def(label_num++, s);

    // Load dispatch table and jump
    emit_load(T1, 2, ACC, s);
    int offset = /* calculated offset */;
    emit_load(T1, offset, T1, s);
    emit_jalr(T1, s);
    emit_addiu(SP, SP, num_params*4, s);
}
```

4.21.1 e.g IV) Object Creation(new syntax)

```
void new__class::code(ostream &s, Environment &env) {
    if (type_name != SELF_TYPE) {
        emit_load_address(ACC, (char*)(string(type_name) + PROTOBJ_SUFFIX).c_str(), s);
        emit_jal("Object.copy", s);
        emit_jal((char*)(string(type_name) + CLASSINIT_SUFFIX).c_str(), s);
    } else {
        // SELF_TYPE handling
        emit_load(T1, 0, SELF, s);
        emit_load_address(T2, CLASSOBJTAB, s);
        emit_load(ACC, 0, T2, s);
        emit_jal("Object.copy", s);
        emit_load(T2, 1, T2, s);
        emit_jalr(T2, s);
    }
}
```

4.22 Challenges and Solutions

- **Inheritance Handling:** Methods are collected recursively to build dispatch tables.
- **Self Type:** Special handling for SELF_TYPE in dispatch and creation.
- **Expression Nesting:** The environment handles nested `let` and `block` structures.
- **Type Checking:** Includes runtime checks for dispatch on `void` and `case` statements.
- **Object Layout:** Attributes use proper offsets respecting inheritance.

4.23 Conclusion

This code generator provides a complete implementation of COOL language features in MIPS assembly, including:

- Object-oriented features (inheritance, dynamic dispatch)
- Primitive types and operations
- Control structures
- Runtime type checking

Testing And Challenges In COOL Compiler Implementation

5.1 Testing

To ensure correctness and stability throughout the development process, I tested my compiler on approximately 70 test cases for each major phase—lexical analysis, parsing, semantic analysis, and code generation. These test cases covered a wide range of input programs, including both valid and erroneous COOL programs, to assess how accurately the compiler detects syntax and semantic errors and generates correct MIPS code.

However, it is important to note that passing a large number of test cases does not guarantee that the compiler is entirely error-free. While empirical testing helps identify bugs and improve reliability, it cannot exhaustively prove the absence of errors. The only rigorous way to establish the correctness of a compiler in all scenarios is through formal verification techniques, which mathematically prove that the compiler adheres to its specification for all possible inputs.

Thus, while my compiler performs correctly on the designed test suite and demonstrates expected behavior across a variety of cases, complete correctness can only be assured by formal methods.

5.2 Challenges

During the development of the compiler, I encountered several challenges, particularly in the code generation phase. One of the most difficult tasks was ensuring that the MIPS assembly code produced by my compiler matched the expected output for each test case. This required establishing semantic equivalence between the generated code and the reference code, which was often thousands of lines long. Verifying correctness in such large outputs was extremely tedious and time-consuming.

To validate the correctness of my generated MIPS code, I frequently resorted to manual dry runs of the assembly instructions and step-by-step tracing to understand the runtime behavior. This was necessary because even minor discrepancies—such as incorrect register usage or stack frame misalignment—could lead to incorrect program execution or runtime crashes.

Moreover, the available MIPS debugging tools posed additional difficulties. While graphical debuggers like QtSPIM and xSPIM offer user-friendly interfaces, they are limited in flexibility and are not well-suited for deep inspection of low-level execution states.

On the other hand, command-line debuggers, though less intuitive, provided more control and precision for examining register values, memory contents, and instruction traces. Learning to effectively use these command-line tools was essential for debugging complex issues in code generation.

Despite these challenges, consistent testing, careful analysis of generated assembly, and iterative refinement of code helped me to gradually resolve most issues. This phase significantly deepened my understanding of both MIPS architecture and the intricacies involved in compiling high-level constructs to low-level code.

Conclusion and Future Work

6.1 Summary of Achievements

This thesis successfully demonstrates the complete design and implementation of a compiler for the COOL (Classroom Object-Oriented Language), covering every major phase of compilation—from lexical analysis and parsing to semantic checking and backend code generation for MIPS assembly.

Through this project, the following key achievements were realized:

- Developed a fully working lexer using Flex, capable of handling all valid and invalid tokens of COOL.
- Implemented a robust parser using Bison that supports COOL grammar, including advanced constructs such as ‘let’, ‘case’, ‘while’, and class inheritance.
- Performed static semantic analysis including type checking, scope resolution, inheritance validation, and method dispatch correctness.
- Designed runtime data structures including class layout tables, dispatch tables, and object initialization sequences.
- Built a manual code generation backend to produce correct and efficient MIPS assembly code for COOL programs.
- Integrated the system with SPIM simulator for testing and debugging the output assembly code.

This project also deepened the author’s understanding of low-level systems programming, runtime memory management, and how high-level abstractions are translated into low-level executable code. Building a compiler from scratch provided hands-on experience with language design, error handling, code emission, and debugging complex semantic issues.

Overall, the project not only met all academic and functional goals but also fulfilled the personal aspiration of understanding compiler internals—a critical stepping stone for contributing to areas such as system software, device driver development, and hardware-software integration in leading technology companies.

6.2 Future Improvements

While the current implementation of the COOL compiler successfully demonstrates the core phases of compilation and supports the essential features of the COOL language, there are several areas where the project can be extended and improved. These enhancements would not only increase the expressiveness and performance of the language but also bring the compiler closer to real-world industrial-grade systems. Some key areas for future work include:

1. **Support for Arrays and Data Structures** The current version of the COOL language does not support arrays or composite data structures such as lists, maps, or tuples. Adding support for arrays would significantly enhance the language's utility by enabling the implementation of algorithms that require indexed data access. This would require changes in the parser, semantic analyzer (type checking, bounds checking), runtime memory layout, and code generator to handle array allocation, indexing, and iteration.
2. **Pure Virtual Functions and Abstract Classes** COOL currently supports inheritance and method overriding, but lacks formal support for abstract classes and pure virtual functions. Introducing this feature would align the language more closely with object-oriented principles seen in modern languages like C++ and Java. It would enable the definition of class interfaces and enforce contract-based programming, requiring subclasses to implement specified methods. Semantic checks would need to be extended to detect unimplemented pure methods and raise compile-time errors appropriately.
3. **Multithreading and Process-Level Parallelism** Adding support for multithreading and processes would open up possibilities for concurrent programming in COOL. This includes thread creation, synchronization primitives (mutexes, semaphores), inter-process communication, and shared memory management. Such features require introducing a runtime system capable of handling thread scheduling, and careful memory model design to avoid data races and undefined behavior. These enhancements would make the language suitable for systems-level and parallel computing tasks.
4. **Machine Learning-based Compiler Optimizations** Compiler optimizations traditionally rely on rule-based heuristics (e.g., constant folding, loop unrolling, dead code elimination). However, modern compiler research increasingly leverages machine learning (ML) to drive optimization decisions. Future versions of the COOL compiler can experiment with ML-guided optimization passes—using reinforcement learning or supervised models trained to predict profitable transformations based on profiling data or code features. For instance, ML models can be trained to choose optimal inlining strategies, register allocation decisions, or loop transformation heuristics.
5. **Integration with LLVM Optimization Passes** Although this compiler currently uses a hand-crafted backend to generate MIPS assembly, integrating with the LLVM toolchain would dramatically expand its optimization capabilities. LLVM provides a rich set of well-tested optimization passes, including but not limited to:

- Constant propagation and folding
- Loop invariant code motion
- Global value numbering
- Dead store elimination
- Interprocedural analysis and optimization
- SSA-based register allocation

Targeting LLVM IR as the compiler’s intermediate representation would also enable support for multiple backends (x86, ARM, WebAssembly), improving portability and performance.

- 6. Garbage Collection and Memory Safety Enhancements** Currently, memory management in COOL is handled manually through basic object allocation and deallocation strategies. Introducing a garbage collector would automate memory reclamation and prevent common memory bugs such as leaks and dangling pointers. This could involve integrating a mark-and-sweep or generational GC model within the runtime, and updating the code generator to emit appropriate hooks.

7. Module System and Separate Compilation

As the COOL language evolves and projects written in it grow in size and complexity, it becomes increasingly important to support modular programming. In the current compiler implementation, all classes and methods are compiled in a monolithic fashion—meaning that every part of the program must be compiled together as a single unit. This is not scalable for larger software development or for enabling reuse across multiple programs.

To address this, a **module system** can be introduced, allowing the language to be split into multiple source files or compilation units. Each module would encapsulate a set of class or function definitions, and support **imports** to include code from other modules. This approach improves both **code organization** and **reusability**, as well as **build efficiency** by allowing unchanged modules to be re-used without recompilation.

Furthermore, the compiler can be extended to support **header files** or interface declarations, similar to ‘.h’ files in C or interface definitions in Java. These headers would declare the types, classes, and functions provided by a module without revealing the full implementation. During compilation, only the interface needs to be analyzed for dependent modules, improving both semantic clarity and compilation speed.

Additional enhancements include:

- **Separate Compilation:** Each ‘.cool’ source file or module can be compiled independently into an intermediate form (e.g., LLVM IR or MIPS assembly), which can later be linked together using a custom linker or backend tool. This supports partial recompilation and drastically reduces build times for large programs.
- **Multi-File Program Support:** Enable the compilation and linking of multiple COOL programs, possibly written by different teams or contributors, into

a single executable. This opens the door to collaborative software development, where libraries and applications are developed separately and composed together.

- **Namespace Support:** Introducing namespaces or module-qualified identifiers prevents name collisions between modules. For example, two modules can define a `print()` method independently, and fully-qualified names like `math.print()` or `debug.print()` resolve ambiguity.
- **Symbol Resolution and Link-Time Checking:** Implementing a proper symbol table management system that spans across modules allows for detection of unresolved or multiply-defined symbols at link time, ensuring consistency and correctness across the program.
- **Import Resolution and Dependency Graphs:** The compiler should be able to resolve `import` or `include` directives, track dependencies between modules, and perform topological sorting of compilation units to respect dependencies.
- **Build System Integration:** Eventually, the compiler can be paired with a simple build tool (like a `Makefile` generator or a COOL-specific build system) that automates dependency tracking, recompilation, and linking.

By adding a module system and enabling separate compilation, the COOL compiler will transition from a toy educational compiler to a more robust infrastructure that mirrors modern compiler architectures. These features are critical for real-world software engineering, where modular design, library reuse, and efficient build processes are indispensable. It also enables the COOL language to serve as a foundation for writing larger and more maintainable applications, further broadening its scope as both an educational tool and a systems-level

8. **Standard Library and File I/O Support** To make COOL practical for real applications, it should include a basic standard library that supports file operations, string utilities, date-time functions, and mathematical computations. Adding file I/O primitives would enable programs to interact with the external environment, read/write data, and perform meaningful system-level tasks.

These improvements aim not only to enrich the COOL language itself but also to align the compiler with real-world, production-grade compilers. By pursuing these directions, the compiler can evolve from an educational project into a powerful platform for experimenting with compiler technologies, language features, and systems-level programming.

Bibliography

- [1] S. University, “Cs143: Compilers course website,” <https://cs143.stanford.edu/>, accessed: 2025-06-04.
- [2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006.
- [3] S. S. Muchnick, *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.

Grammar Rules

The grammar is presented using extended BNF notation where:

- A^* means zero or more A 's
- A^+ means one or more A 's
- Items in square brackets [...] are optional
- Double brackets [] group grammar symbols

```
program ::= [class]^+
```

```
class ::= class TYPE [inherits TYPE] { [_feature_]~* }
```

```
feature ::= ID( [_formal_[,formal]^* ] ) : TYPE { _expr_ }  
          | ID : TYPE [ <- _expr_ ]
```

```
formal ::= ID : TYPE
```

```
expr ::= ID <- _expr_  
        | _expr_[@TYPE].ID( [_expr_[,_expr_]~* ] )  
        | ID( [_expr_[,_expr_]~* ] )  
        | if _expr_ then _expr_ else _expr_ fi  
        | while _expr_ loop _expr_ pool  
        | { [_expr_;]^+ }  
        | let ID : TYPE [ <- _expr_ ] [,ID : TYPE [ <- _expr_ ] ]~* in _expr_  
        | case _expr_ of [ID : TYPE => _expr_;]^+ esac  
        | new TYPE  
        | isvoid _expr_  
        | _expr_ + _expr_  
        | _expr_ - _expr_  
        | _expr_ * _expr_  
        | _expr_ / _expr_  
        | ~ _expr_  
        | _expr_ < _expr_  
        | _expr_ <= _expr_  
        | _expr_ = _expr_  
        | not _expr_
```

```
| (_expr_)  
| ID  
| integer  
| string  
| true  
| false
```

Operator Precedence

Operator precedence from highest to lowest:

1. @
- 2.
3. isvoid
4. * /
5. + -
6. <= < =
7. not
8. <-

- All binary operations are left-associative except:
- Assignment (<-) is right-associative
- Comparison operations (<=, <, =) do not associate

Sample Programs

A.1 Binary Search Tree

```
class Node inherits Object {
  value : Int;
  next : Node;

  init(val : Int, nxt : Node) : SELF_TYPE {
    {
      value <- val;
      next <- nxt;
      self;
    }
  };

  get_value() : Int {
    value
  };

  get_next() : Node {
    next
  };

  set_next(nxt : Node) : SELF_TYPE {
    {
      next <- nxt;
      self;
    }
  };
};

class LinkedList inherits IO {
  head : Node;

  init() : SELF_TYPE {
    {
      head <- new Node;
    }
  };
};
```

```

        head <- head.init(0, new Node);
        self;
    }
};

add_front(val : Int) : SELF_TYPE {
    {
        let new_node : Node <- new Node in {
            new_node <- new_node.init(val, head.get_next());
            head.set_next(new_node);
            self;
        };
    }
};

print_all() : Object {
    {
        let curr : Node <- head.get_next() in
        while not isvoid(curr) loop {
            (out_int(curr.get_value())).out_string("\n");
            curr <- curr.get_next();
        } pool;
    }
};

};

class Main inherits IO {
    main() : Object {
        {
            let list : LinkedList <- new LinkedList,
                i : Int <- 3,
                x : Int in
            {
                list.init();

                while not (i <= 0) loop
                {
                    x <- in_int();
                    list.add_front(x);
                    i <- i - 1;
                }
                pool;

                list.print_all();
            };
        }
    };
};

```

```
};
```

A.2 Linked List

```
class BSTNode inherits Object {
  value : Int;
  left : BSTNode;  -- defaults to void (uninitialized)
  right : BSTNode; -- defaults to void

  init(v : Int) : SELF_TYPE {
    {
      value <- v;
      -- left and right remain uninitialized, i.e., void
      self;
    }
  };

  insert(v : Int) : SELF_TYPE {
    {
      if v < value then
        if isvoid(left) then
          left <- (new BSTNode).init(v)
        else
          left.insert(v)
        fi
      else
        if isvoid(right) then
          right <- (new BSTNode).init(v)
        else
          right.insert(v)
        fi
      fi;
      self;
    }
  };

  print_in_order() : Object {
    {
      if not isvoid(left) then
        left.print_in_order()
      else 0 fi;
      (new IO).out_int(value).out_string("\n");
      if not isvoid(right) then
        right.print_in_order()
      else 0 fi;
      0;
    }
  }
}
```

```

    };
};

class BSTree inherits Object {
    root : BSTNode; -- defaults to void

    init() : SELF_TYPE {
        {
            -- no need to assign root to void; it's already void by default
            self;
        }
    };

    insert(v : Int) : SELF_TYPE {
        {
            if isvoid(root) then
                root <- (new BSTNode).init(v)
            else
                root.insert(v)
            fi;
            self;
        }
    };

    print_all() : Object {
        {
            if not isvoid(root) then
                root.print_in_order()
            else
                (new IO).out_string("Tree is empty.\n")
            fi;
            0;
        }
    };
};

```

```

class Main inherits Object {
    main() : Object {
        {
            let tree : BSTree <- (new BSTree).init() in {
                tree.insert(5);
                tree.insert(2);
                tree.insert(8);
                tree.insert(1);
                tree.insert(3);
                tree.print_all();
            }
        }
    }
}

```

```

    };
  }
};

```

A.3 Bubble Sort

```

class Node {
  value : Int;
  prev : Node;
  next : Node;

  init(v : Int) : SELF_TYPE {{
    value <- v;
    self;}}
};

set_prev(p : Node) : Object {{
  prev <- p;
  0;}}
};

set_next(n : Node) : Object {{
  next <- n;
  0;}}
};

get_prev() : Node { prev };
get_next() : Node { next };
get_value() : Int { value };
set_value(v : Int) : Object { value <- v };
};

class List {
  head : Node;
  tail : Node;

  add_node(v : Int) : Object {
    let new_node : Node <- (new Node).init(v) in
    if isvoid head then {
      head <- new_node;
      tail <- new_node;
    } else {
      tail.set_next(new_node);
      new_node.set_prev(tail);
      tail <- new_node;
    }
  }
}

```

```

        fi

};

print_list() : Object {
    let curr : Node <- head in {
        while not isvoid curr loop {
            (new IO).out_int(curr.get_value()).out_string(" ");
            curr <- curr.get_next();
        } pool;
        (new IO).out_string("\n");
    }
};

bubble_sort() : Object {
    let swapped : Bool <- true, curr : Node, tmp : Int in
        while swapped loop {
            swapped <- false;
            curr <- head;
            while not isvoid curr.get_next() loop
                if curr.get_next().get_value() < curr.get_value() then {
                    tmp <- curr.get_value();
                    curr.set_value(curr.get_next().get_value());
                    curr.get_next().set_value(tmp);
                    swapped <- true;
                } else
                    curr <- curr.get_next()
                fi
            pool;
        } pool
};

};

class Main inherits IO {
    main() : Object {
        let lst : List <- new List in {
            lst.add_node(4);
            lst.add_node(2);
            lst.add_node(5);
            lst.add_node(1);
            lst.add_node(3);
            out_string("Before Sorting:\n");
            lst.print_list();
            lst.bubble_sort();
            out_string("After Sorting:\n");
            lst.print_list();

```

```
    }  
};  
};
```